

TP previo Parcial 2 DDS 2026 - Curso 3K5

Campo	Valor
Materia	Desarrollo de Software (DDS)
Año	2026
Instancia de evaluación	TP previo - Parcial 2
Documento	Enunciado de trabajo práctico
Curso	3K5

TP previo Parcial 2 DDS 2026 - Curso 3K5

Introducción

Desde la publicación de la tarea hasta el sábado 13/06 a las 23:55 hs. Cada grupo de entre 2 y 5 alumnos va a desarrollar una aplicación full stack para agenda de entrevistas. El sistema debe coordinar postulantes, entrevistadores, horarios, modalidad, reprogramaciones y resultados, evitando superposiciones y respetando permisos. El objetivo es modelar un proceso de selección completo, con reglas de negocio y seguimiento, no una agenda plana. Adicionalmente, cada alumno deberá subir el trabajo realizado a su portafolio.

Desde el 14/6 10 hs hasta el lunes 15/6 a las 23:55 otro grupo deberá realizar una revisión / corrección entre pares (revisión entre grupos) y entregar la devolución grupal.

Estas dos tareas son esenciales y previas al segundo parcial ya que habrá 2 preguntas a desarrollar relativas al tema desarrollado.

Este práctico integra los contenidos principales del segundo parcial: Express Router, middlewares, manejo centralizado de errores, CORS, autenticación y autorización con JWT, pruebas de API con Jest y Supertest, React con Vite, componentes, estado, efectos, React Router, formularios, Axios, Validaciones y manejo de errores de API.

Como el trabajo se resuelve en grupo y con una semana de desarrollo, se espera una solución de complejidad media-alta: reglas de negocio reales, estados, permisos, historial, filtros avanzados, persistencia, datos semilla, pruebas automatizadas y una interfaz navegable que permita operar el sistema completo.

Alcance mínimo de resolución grupal

La entrega debe evidenciar que hubo división de responsabilidades y posterior integración. Como mínimo, el sistema debe incluir:

- un módulo de autenticación completo, con registro, login, JWT, roles y persistencia de sesión en frontend;
- un módulo principal del dominio con listado, filtros, detalle, alta, edición y acciones de estado;

- un módulo de administración o resumen que muestre información agregada del dominio;
- Validaciones de negocio implementadas en servicios del backend, no solamente en formularios;
- al menos dos flujos end-to-end funcionando desde la interfaz hasta la persistencia;
- pruebas automatizadas que cubran casos exitosos, errores de validación, errores de permisos y reglas específicas del dominio;
- documentación suficiente para que otro grupo pueda ejecutar, probar y explicar la solución.

No se considera suficiente una entrega que solo tenga pantallas sin backend real, endpoints sin frontend integrado, datos hardcodeados en componentes, o reglas de negocio aplicadas únicamente de forma visual.

Qué tienen que construir

Desarrollar una **aplicación web full stack para agenda de entrevistas**. La aplicación debe permitir **programar entrevistas para postulantes, asignar entrevistador y evitar superposiciones de horario**. No es un CRUD genérico: cada entrevista debe validar disponibilidad real del entrevistador y estado del postulante.

En el parcial presencial cada estudiante responderá preguntas escritas sobre la solución entregada por su grupo. Deben poder explicar rutas, componentes, servicios, middlewares, Validaciones, JWT, permisos, pruebas y errores manejados.

Dominio obligatorio

El sistema trabaja sobre el dominio de agenda de entrevistas. El recurso principal que se administra es la **Entrevista: una instancia programada entre un postulante y un entrevistador, con fecha, horario, modalidad, estado y observaciones**. Cada entrevista debe estar asociada obligatoriamente a un Postulante, que representa a la persona que participa del proceso de selección.

La regla central del dominio es que **una entrevista solo puede programarse o reprogramarse si el postulante existe y sigue en proceso, el entrevistador no tiene otra entrevista superpuesta y los datos de modalidad son consistentes con el tipo elegido.**

Entidades mínimas obligatorias

El sistema debe modelar y persistir al menos 4 entidades. Si usan archivo JSON, SQLite, Sequelize u otra alternativa, igual deben respetar esta separación lógica:

Entidad	Para qué existe	Campos mínimos	Relaciones principales
usuarios	Representa entrevistadores,	id, nombre, email,	Un usuario puede ser

	RRHH y administradores que intervienen en el proceso.	passwordHash, rol, activo	entrevistador asignado y generar historial.
postulantes	Representa personas candidatas dentro del proceso de selección.	id, nombre, apellido, email, teléfono, puesto, estado	Un postulante puede tener muchas entrevistas.
entrevistas	Representa una instancia programada entre postulante y entrevistador.	id, postulanteId, entrevistadorId, fecha, horaInicio, horaFin, modalidad, ubicación o link, estado, observaciones	Pertenece a un postulante y tiene un usuario entrevistador.
historial_entrevistas	Registra auditoría de cambios sobre entrevistas.	id, entrevistaId, usuarioId, acción, fechaHora, valorAnterior, valorNuevo	Pertenece a una entrevista y al usuario que hizo la acción.

Campos mínimos por entidad

Los campos pueden adaptarse al motor de persistencia elegido, pero la información y las relaciones deben estar presentes.

Usuario

Campo	Tipo sugerido	Obligatorio	Descripción	Ejemplo o valores posibles
id	número/string	Sí	Identificador único del usuario.	1, usr-001
nombre	string	Sí	Nombre visible del usuario.	Sofía Luna
email	string	Sí	Credencial de acceso y dato de contacto. Debe ser único.	sofia@dds.com
passwordHash	string	Sí	Contraseña almacenada de forma segura o hash simulado documentado.	bcrypt_hash

rol	string	Sí	Define permisos dentro del sistema.	entrevistador, rrhh, admin
activo	boolean	Sí	Permite bloquear accesos sin borrar el usuario.	true, false

Postulante

Campo	Tipo sugerido	Obligatorio	Descripción	Ejemplo o valores posibles
id	número/string	Sí	Identificador único del postulante.	post-001
nombre	string	Sí	Nombre del postulante.	Juan
apellido	string	Sí	Apellido del postulante.	Martínez
email	string	Sí	Contacto principal. Debe ser único si el grupo lo decide.	juan@mail.com
teléfono	string	No	Contacto telefónico.	3515551234
puesto	string	Sí	Puesto o búsqueda a la que aplica.	Frontend Junior
estado	string	Sí	Estado dentro del proceso.	nuevo, en_proceso, rechazado, contratado

Entrevista

Campo	Tipo sugerido	Obligatorio	Descripción	Ejemplo o valores posibles
id	número/string	Sí	Identificador único de la entrevista.	ent-1001
postulanteId	número/string	Sí	Postulante entrevistado. Debe existir en postulantes.	post-001
entrevistadorId	número/string	Sí	Usuario asignado como entrevistador.	usr-entrev-1
fecha	date/string	Sí	Día programado.	2026-06-18

horaInicio	string	Sí	Inicio de la entrevista.	14:00
horaFin	string	Sí	Fin de la entrevista. Debe ser mayor que horaInicio.	14:45
modalidad	string	Sí	Forma en que se realiza.	presencial, virtual
ubicación o link	string	Sí	Lugar físico o enlace, según modalidad.	Sala 2, https://meet...
estado	string	Sí	Estado actual del flujo.	programada, realizada, cancelada, reprogramada
observaciones	string	No	Comentarios del entrevistador o RRHH.	Buen manejo técnico

HistorialEntrevista

Campo	Tipo sugerido	Obligatorio	Descripción	Ejemplo o valores posibles
id	número/string	Sí	Identificador único del registro de auditoría.	hist-001
entrevistaId	número/string	Sí	Entrevista afectada por el cambio.	ent-1001
usuarioId	número/string	Sí	Usuario que realizó la acción.	usr-rrhh
acción	string	Sí	Operación realizada sobre la entrevista.	creación, edición, reprogramación, cancelación, realización
fechaHora	datetime/string	Sí	Momento exacto de la acción.	2026-06-08T11:15:00
valorAnterior	object/string	No	Datos relevantes antes del cambio.	{ "fecha": "2026-06-18" }
valorNuevo	object/string	No	Datos relevantes	{ "fecha": "2026-06-19" }

			después del cambio.	
--	--	--	---------------------	--

Reglas funcionales específicas

La aplicación debe resolver estos casos:

1. Registrar usuario e iniciar sesión.
2. Obtener un JWT al iniciar sesión correctamente.
3. Listar entrevistas con filtros combinables por fecha, estado, entrevistadorId y postulanteId.
4. Ver el detalle de una entrevista desde una ruta dinámica.
5. Crear una entrevista seleccionando un postulante existente.
6. Editar fecha, horario, modalidad, ubicación o estado según permisos.
7. Cancelar una entrevista cambiando estado a cancelada.
8. Rechazar entrevistas si horaInicio no es menor que horaFin.
9. Rechazar entrevistas si el entrevistador ya tiene una entrevista programada o reprogramada en una franja superpuesta.
10. Rechazar entrevistas para postulantes en estado rechazado o contratado.

Complejidad adicional obligatoria

El grupo debe implementar también:

1. Flujo completo de estados: programada -> realizada o cancelada; programada -> reprogramada; reprogramada -> realizada o cancelada. No permitir modificar entrevistas realizada salvo observaciones.
2. Búsqueda paginada y ordenable: aceptar page, limit, sortBy y order en el listado de entrevistas.
3. Vista resumen para administración con entrevistas del día, entrevistas por entrevistador, postulantes en proceso y entrevistas canceladas.
4. Historial de cambios de cada entrevista: guardar fecha, usuario, acción y valores modificados cuando se reprograma, cancela, realiza o edita.
5. Validación de modalidad: si es virtual debe tener link; si es presencial debe tener ubicación.
6. Control de superposición también al reprogramar o editar horario, ignorando la propia entrevista pero comparando contra las demás.
7. Semilla de datos inicial con al menos 8 postulantes, 3 entrevistadores, 1 admin/rrhh y 12 entrevistas en distintos estados.
8. Mensajes de error distintos para postulante inexistente, postulante no elegible, entrevistador ocupado, horario invalido y falta de permisos.

Aclaraciones para resolver el alcance

Estas aclaraciones responden dudas esperables al momento de implementar:

Que postulantes pueden entrevistarse? Solo postulantes en estado nuevo o en_proceso. No se deben programar entrevistas para rechazado o contratado.

Que entrevistas bloquean horario del entrevistador? Las entrevistas programada y reprogramada del mismo entrevistador y fecha.

Como se valida modalidad? Si la modalidad es virtual, debe existir link; si es presencial, debe existir ubicación.

Que significa reprogramar? Cambiar fecha, horario o entrevistador, registrar historial y dejar la entrevista en estado reprogramada.

Que debe mostrar el resumen? Como mínimo: entrevistas del día, entrevistas por entrevistador, postulantes en proceso y entrevistas canceladas.

Donde se valida la superposicion? En el servicio del backend, tanto al crear como al editar o reprogramar.

Roles y permisos

Deben existir al menos estos roles:

entrevistador: puede ver entrevistas asignadas, marcar como realizada y agregar observaciones.

admin o rrhh: puede crear, reprogramar, cancelar y ver todas las entrevistas.

Al menos una ruta de escritura debe devolver:

401 si no se envía JWT.

403 si el usuario está autenticado pero no tiene permiso para la acción.

Backend esperado

Implementar backend con Node.js y Express.

Rutas mínimas:

- POST /api/auth/register
- POST /api/auth/login
- GET /api/postulantes
- GET /api/entrevistas?fecha=&estado=&entrevistadorId=&postulanteId=
- GET /api/entrevistas/resumen
- GET /api/entrevistas/:id
- GET /api/entrevistas/:id/historial
- POST /api/entrevistas
- PUT /api/entrevistas/:id
- PATCH /api/entrevistas/:id/cancelar
- PATCH /api/entrevistas/:id/realizar
- PATCH /api/entrevistas/:id/reprogramar

Estructura minima:

- routes/entrevistas.routes.js con express.Router()
- controlador de entrevistas

- servicio de entrevistas con regla de superposicion por entrevistador
- middleware de autenticación JWT
- middleware de autorización por rol o entrevistador asignado
- middleware de validación de entrada
- middleware de manejo de errores con firma (err, req, res, next)
- persistencia en archivo JSON, SQLite, Sequelize u otra solución que conserve datos al reiniciar el

backend

- carga de datos semilla para probar el dominio sin cargar todo manualmente

Usar status HTTP coherentes: 200, 201, 400, 401, 403, 404 y 500. Las respuestas de error

deben tener JSON claro, por ejemplo { "error": "El entrevistador ya tiene una

entrevista en ese horario" }.

Frontend esperado

Implementar frontend con React y Vite.

Pantallas mínimas:

1. Login y registro.
2. Listado de entrevistas con filtros por fecha, estado, entrevistador y postulante.
3. Detalle de entrevista en una ruta como /entrevistas/:id.
4. Pantalla transaccional de alta/edición o reprogramación de entrevista, donde se selecciona postulante, entrevistador, fecha, horario y modalidad, y se confirma la operación contra la API.
5. Acciones visibles para cancelar, realizar o reprogramar según rol.
6. Panel resumen para administradores o RRHH.
7. Historial visible dentro del detalle de una entrevista.
8. Ruta comodín para página no encontrada.

La capa de servicios debe usar Axios:

- instancia con baseURL
- params para filtros
- body en POST, PUT o PATCH
- header Authorization: Bearer <token> en acciones protegidas
- manejo visible de errores de validación, autenticación, autorización y recurso inexistente
- componentes separados para tabla/listado, filtros, formulario, detalle, acciones por rol y resumen administrativo

- estados de carga, vacío, error y éxito para las operaciones principales

Integración y calidad esperada

La resolución debe mostrar trabajo de equipo y no una acumulación de archivos sueltos. Se espera:

- contexto, hook o mecanismo equivalente para conservar usuario autenticado, token y rol en frontend;
- rutas protegidas en frontend y backend, no solo botones ocultos;
- Validaciones repetidas en frontend para experiencia de usuario y en backend como fuente de verdad;
- servicios Axios separados por recurso, sin llamadas HTTP mezcladas dentro de todos los componentes;
- contraseñas hashadas o, si usan usuarios semilla simplificados, aclaración explícita de la limitación en el README;
- payload del JWT sin contraseñas ni datos sensibles;
- manejo de errores centralizado en backend y mensajes comprensibles en frontend;
- decisión documentada sobre persistencia, estructura de carpetas y división de responsabilidades del grupo.

Testing mínimo

El backend debe incluir pruebas con Jest y Supertest para:

1. Login correcto e inválido.
 2. Listado de entrevistas con y sin filtros.
 3. Detalle de entrevista existente e inexistente.
 4. creación válida de una entrevista.
 5. creación inválida por horario inconsistente.
 6. creación inválida por superposición del entrevistador.
 7. Acceso sin JWT a una ruta protegida.
 8. Acceso con JWT de entrevistador a una acción solo permitida para admin o rrhh.
 9. Reprogramación inválida por superposición del entrevistador.
 10. Validación de modalidad virtual sin link o presencial sin ubicación.
- Las pruebas deben verificar status HTTP y cuerpo JSON relevante.

Documentación obligatoria

La entrega debe incluir un README.md con:

- instrucciones para ejecutar backend y frontend
- usuario admin o rrhh y usuario entrevistador de prueba
- listado de endpoints principales
- listado de rutas frontend
- Validación de superposicion por entrevistador y estados del postulante
- Validación de JWT, roles y permisos
- comando para ejecutar pruebas
- limitaciones conocidas

Preguntas frecuentes y criterios de corrección

Sobre alcance y evaluación

El TP es condición de entrega y también será la base de preguntas abiertas del Parcial 2. La nota del parcial surge del examen, pero una entrega incompleta o no explicable afecta directamente las respuestas escritas.

En el parcial se preguntará sobre el código real entregado por el grupo: rutas, servicios,

- componentes, Validaciones, JWT, permisos, pruebas y decisiones.

Se aceptan soluciones equivalentes si respetan entidades mínimas, reglas del dominio, rutas, pantallas, pruebas y criterios de seguridad. Cualquier cambio importante debe estar justificado en el README.

La persistencia puede ser archivo JSON, SQLite, Sequelize u otra alternativa, siempre que conserve datos al reiniciar y respete las entidades del enunciado.

Debe haber registro/login real desde frontend. Los usuarios semilla son obligatorios para probar rápido, pero no reemplazan el flujo de autenticación.

El historial debe existir en backend y verse desde frontend, al menos en el detalle del recurso principal.

La pantalla transaccional puede ser una sola pantalla para alta/edición o dos pantallas separadas, pero debe confirmar la operación contra la API y mostrar errores.

El resumen administrativo debe estar protegido para roles administrativos. Filtros, paginacion y ordenamiento deben resolverse en backend. El frontend solo envía params y muestra resultados.

Documentar una limitacion no reemplaza un requisito mínimo. Sirve para explicar decisiones o faltantes menores, no para omitir reglas centrales.

Sobre backend

Deben usar `express.Router()` en archivos separados para el recurso principal.

El middleware de errores con firma (`err, req, res, next`) debe quedar después de las rutas.

Los errores de validación deben responder con status coherente, normalmente 400, y JSON claro, por ejemplo `{ "error": "mensaje" }`.

La autorización debe validar rol y, cuando corresponda, propiedad del recurso. No alcanza con que el JWT exista.

El JWT no debe incluir contraseñas ni datos sensibles.

Las contraseñas deben guardarse hasheadas; si se usa una simplificación por tiempo, debe estar documentada y no exponer contraseñas en respuestas.

Las reglas de negocio deben estar en servicios del backend, no solamente en controladores ni formularios.

Deben estar protegidas las acciones de escritura y cualquier lectura privada o administrativa.

Sobre frontend

React Router debe incluir ruta comodín `*`.

Las pantallas de detalle deben leer parámetros con `useParams`.

Pueden usar React Hook Form o formularios controlados con `useState`, pero las Validaciones deben verse en pantalla.

Los errores de API deben mostrarse de forma comprensible cerca de la acción que fallo o en un área visible de la pantalla.

Axios debe estar en una capa de servicios separada. No mezclar llamadas HTTP en todos los componentes.

El JWT puede enviarse con interceptores o agregando headers en servicios, pero siempre como `Authorization: Bearer <token>`.

Proteger una ruta en frontend significa impedir acceso visual/navegable si no hay usuario autenticado o rol suficiente; esto no reemplaza la protección del backend.

Sobre testing

Se exigen pruebas de backend con Jest y Supertest. Tests de frontend son opcionales.

Cada test debe validar status HTTP y cuerpo JSON relevante.

Deben probar login correcto, login invalido, acceso sin token y acceso con rol insuficiente.

Deben probar reglas específicas del dominio, no solo endpoints felices.

Los datos de prueba deben ser previsibles. Si un test modifica datos, deben resetearlos o aislarlos para no afectar otros tests.

Entrega

Subir a Moodle el repositorio o archivo comprimido en el portafolio del alumno y en la entrega grupal. No incluir carpetas node_modules. Sí deben incluir package.json, código fuente, tests y documentación.